COMP101
Introduction to Programming
2025-26

Lecture-22
Data Collections - Lists

1

Data Collections

- We usually have one data item assigned to one variable

  name = ‘stephen’

- It is possible to assign several data items to one single variable if we identify a connection between those data items
  - stephen, leszek, penny, victor, jenny
    - they are siblings (brothers and sisters)
    - or students on a module etc

Data Collections

- Four data types can store multiple values
- List []
- Tuple ()
- Set {}
- Dictionary {:}

Often referred to as ‘array’: These are ‘one-dimensional arrays’ (‘1-d arrays’)

They are ‘data types’ just as int, float etc are data types and are used to store several (related) data items (‘elements’) using just one var name and indexing

Data Collections – list[]

- siblingList =[‘stephen’,’leszek’,’penny’,’victor’,’jenny’]
- note brackets
- ordered (has an index, start=0)
- mutable elements – can be changed
- duplicate elements allowed
- data type can be mixed
- use when lot of adds/amends/deletes
  - Can be slow for access and iteration

Data Collections – tuple()

- siblingTuple =('stephen','leszek','penny','victor','jenny')
- note brackets
- ordered (has an index, start=0)
- immutable elements – can't be changed
- duplicate elements allowed
- data types can be mixed
- use for fixed data types not to be changed
  - faster than a list especially for large volumes

Data Collections – set{}

- siblingSet={‘stephen’,’leszek’,’penny’,’victor’,’jenny’}
- note brackets
- unordered (no index)
- immutable elements
- duplicate elements not allowed
- data types can be mixed
- Use to check the data set for an item
  - very efficient checking

Data Collections – dictionary{:}

- siblingDict={‘stephen’:’male’, ‘leszek’:’male’, ‘penny’:female’, ‘victor’:’male’, ‘jenny’:female’}
- from Python v.3.7 and above
- ‘associative array’ of ‘key:value pairs’
- ordered (has an index)
- mutable, no duplicates
- fast processing for large volumes of data
- retrieve data via its key

List indexing and access

- Each element has an index starting at [0]

siblingList =[‘stephen’,’leszek’,’penny’,’victor’,’jenny’]

[0] [1] [2] [3] [4]

print siblingList[2]

penny

Access (‘reference’) to an item is via its index;
Index always starts at [0] and are always integer – show them in brackets;

– can’t access individual characters as only the full names associate with an index - not the individual letters in them

List indexing and access

Can change a name:

#change penny to benny

siblingList =[‘stephen’,’leszek’,’penny’,’victor’,’jenny’]

[0] [1] [2] [3] [4]

#access the full element to make an amendment – lists are mutable

siblingList[2] = ‘benny’

print(siblingList)

[‘stephen’,’leszek’,’benny’,’victor’,’jenny’]

List to string conversion – ‘ ‘.join()

- To access individual characters in an element, we can convert the list into a string and each character has an index number
- ‘ ‘.join(name of the list)

  .join() is preceded with a space in apostrophes
  - each element will be separated by a space
  - if there was a comma between the apostrophes we get a comma between each element (CSV) etc
  - ‘instance method’
- Only works with strings

List to string - .join()

#convert the list of 5 elements into a string of 32 elements with .join()

siblingList = ['stephen', 'leszek', 'penny', 'victor', 'jenny']

[0]	[1]	[2]	[3]	[4]

siblingString = ' '.join(siblingList) #space separator

print (siblingString)

stephen leszek penny victor jenny #32 characters - each has an index

[0][1][2]…… …….. [32] - every character has an index number

siblingString = ','.join(siblingList) #comma separator CSV

print (siblingString)

stephen,leszek,penny,victor,jenny

String to List - .split()

Information only

- Problem: .join() has created a string
  - but strings are immutable (can't be changed)
- We can convert the string back to a list with a 'split string function'

```
var = string_name.split(separator)
```

- This gives us back access to change characters within individual elements

String to List - .split()

Information only

```
#siblingList has 5 indexes
siblingList = ['stephen', 'leszek', 'penny', 'victor', 'jenny']
['stephen', 'leszek', 'penny', 'victor', 'jenny']

We can .join() it to turn it all into one string with 32 indexes
siblingString = ' '.join(siblingList)
stephen leszek penny victor jenny

# .split() to turn it back into a list with 5 indexes
splitList = siblingString.split( )
['stephen', 'leszek', 'penny', 'victor', 'jenny']
```

List ops: output by ‘slicing’

siblingList = ['stephen', 'leszek', 'penny', 'victor', 'jenny']

[0] [1] [2] [3] [4]

#slice specific elements – last index is excluded

print (siblingList[1:3])

[‘leszek’, ‘penny’]

print (siblingList[0:5]) #extend the slice range to o/p all names

[‘stephen’, ‘leszek’, ‘penny’, ‘victor’, ‘jenny’]

Use an index range to slice out part of the list
[1:2] start at index[1] and stop at index[2] ’leszek’ index[2] does not o/p
[0:4] start at index[0] and stop at index[4] ‘stephen’, ‘leszek’, ‘penny’, ‘victor’
Last index does not print out from the slice
If we want it – extend the slice range:

List ops: output by slicing

```
siblingList = ['stephen', 'leszek', 'penny', 'victor', 'jenny']
                [0]       [1]       [2]       [3]       [4]

#slice from start of list to a given index
print (siblingList[:3]) #default start is [0]
    ['stephen', leszek', 'penny']

#slice from a given index to the end of the list
print (siblingList[3:])
    ['victor', 'jenny']
```

List ops: slicing with stride

```
siblingList = ['stephen', 'leszek', 'penny', 'victor', 'jenny']
                  [0]       [1]      [2]       [3]      [4]
#use a 'stride':  start [0:5] is start[0] and process all five names
#[0:5:2] is start at [0] and process all five names with a stride ('step') of 2
print (siblingList[0:5:2])
    ['stephen', 'penny', 'jenny']

#[::2] defaults as start at [0] and process all five names taking every second element
print (siblingList[::2])
    ['stephen', 'penny', 'jenny']
```

Extending the list

- siblingList has names of brothers and sisters
  - but they call themselves by short-names
- stephen is ‘ste’
- leszek is ‘les’
- penny is ‘pen’
- victor is ‘vic’
- jenny is ‘jen’

Extending the list

- We can create a new list that uses these short-names
- This is achieved by ‘slicing’
  - slice out parts of the name and put the slice into a new list
    - 1. Create a new list
    - 2. for each name, slice the first 3 chars
    - 3. append the 3 chars to the new list

List slicing

```
siblingList = ['stephen', 'leszek', 'penny', 'victor', 'jenny']
#initialise an empty list to hold first 3 characters of the name
familiarList = []
#loop through every name in the original list
for i in (siblingList): #loop 5 times
        familiarList.append(i[:3]) #slice first 3 letters from each  name

print (familiarList)
['ste', 'les', 'pen', 'vic', 'jen'] #append adds to the end of the list
```

List slicing

- We now have 2 lists:

#siblingList

['stephen', 'leszek', 'penny', 'victor', 'jenny']

#familiarList

['ste', 'les', 'pen', 'vic', 'jen']

- We can concatenate these lists to produce a third list showing name and short-name

#sibling-familiarList

[‘stephen ste', 'leszek les', 'penny pen', 'victor vic', 'jenny jen']

List slicing and concatenation

```
siblingFamiliar = []                    #third list

for i in range (len(siblingList)): #loop 5 times through both lists
    firstFull = siblingList[i]              #full name from first list
    secondFamiliar = familiarList[i] #3-char name from short-name list
#concatenate full name with short-name with a space between them
    siblingName = firstFull + ' ' + secondFamiliar
    siblingFamiliar.append(siblingName)

print (siblingFamiliar)
['stephen ste', 'leszek les', 'penny pen', 'victor vic', 'jenny jen']
```

List concatenation ‘+’

```
#initialise two lists and concatenate with ‘+’ operator
listNum = [1,2,3]
listStr = ["a","b","c"]

combined = listNum + listStr     #concatenate
print(combined)
[1, 2, 3, 'a', 'b', 'c']
```

List ops: create & populate

- Create a list (empty)

```
spectrumList = [ ]
```

- Populate a list

```
spectrumList = [“red”, “orange”, “green”, “blue”]
```

Items can be any type and can be mixed types.
Here they are all string-type

List ops: output

spectrumList = [“red”, “orange”, “green”, “blue”]

- Output the full list

```
print (spectrumList)
['red', 'orange', 'green', 'blue']
```

Output is single quotes around strings (but not around numbers)

Outputs brackets also

- Output one item – use its index

```
print (spectrumList [2])
green        #green is at index[2]
```

Output has no quotes around single string, no brackets

List ops: output by ‘slicing’

spectrumList = [“red”, “orange”, “green”, “blue”]

[0] [1] [2] [3]

- Output several items

  print (spectrumList [1:3])

  [‘orange’, ‘green’]

[1:3] is called ‘slicing’.
Output items in list starting at index [1] and stop at index [3]

Last item does not print out from the slice. If we want it – extend the slice range

- Output several items including the last one

  print (spectrumList [1:4])

  [‘orange’, ‘green’, ‘blue’]

List ops: add items by index

spectrumList = [“red”, “orange”, “green”, “blue”]

Add by index number (anywhere in list)

– yellow should be after orange and before green

#[2:2] means ‘add at index [2]’

spectrumList[2:2] = [“yellow”]

print (spectrumList)

[‘red’, ‘orange’, ‘yellow’, ‘green’, ‘blue’]

Can add several items, comma separated

List ops: add items by insert

.insert() anywhere in list

```
spectrumList = [“red”, “orange”, “green”, “blue”]
#add ‘colours’ to start of list
spectrumList.insert (0, ’colours’) #insert at index[0]

print (spectrumList)
[‘colours’, ‘red’, ‘orange’, ‘yellow’, ‘green’, ‘blue’]
```

List ops: add items by append

.append() only places item at end of list

```
spectrumList =[‘red’, ‘orange’, ‘yellow’, ‘green’, ‘blue’]
# indigo should go after blue
spectrumList.append (“indigo”)

print (spectrumList)
[‘red’, ‘orange’, ‘yellow’, ‘green’, ‘blue’, ‘indigo’]
```

List ops: delete items by ‘del’

spectrumList =[‘red’, ‘orange’, ‘yellow’, ‘green’, ‘blue’, ‘indigo’]

‘del’ uses index value

```
# delete ‘orange’
del spectrumList [1]
print (spectrumList)
[‘red’, ‘yellow’, ‘green’, ‘blue’, ‘indigo’]

# delete ‘green’ and ‘blue’
del spectrumList [2:4]
print (spectrumList)
[‘red’, ‘yellow’, ‘indigo’]
```

Parameter for the index must be an integer
Can delete several items at same time

Parameter for the index range must be a slice.
The slice range will delete index [2] and index [3]

List ops: delete item by ‘remove’

spectrumList = [‘red’, ‘orange’, ‘yellow’, ‘green’, ‘blue’, ‘blue’ ‘indigo’]

‘remove’ uses item value

If the colour is not in list we get an error message

colourToDelete = input (“Enter colour: “) blue

spectrumList.remove(colourToDelete)

print (spectrumList)

[‘red’, ‘orange’, ‘yellow’, ‘green’, ‘blue’ ‘indigo’]

Take a look at the list assignment at the top of the slide - ‘blue’ is in twice:
.remove() deletes the first occurrence, but leaves any others after it

List ops: len and sum

- Count items – len( )

```
spectrumList = [“red”, “orange”, “green”, “blue”]
print (len(spectrumList)
4
```

- Sum numeric items – sum( )

```
numList = [1,2,3]
total = sum(numList)
print (total)
6
```

Lists Summary

User-defined name = [data comma-separated]

num_list = [1,2,3,4]

animal_list = [“cat”, “dog”, “cow”, “big bird”]

Lists are ordered (have an index) but samples below are not the same – they have the same data items but in a different order

num_list1 = [1,2,3,4]

num_list2 = [3,1,4,2]

num_list1 index[0]=1

num_list2 index[0]=3
Hence not the same list

List summary

- List items associate with an automatic index

num_list = [1, 2, 3, 4]

[0] [1] [2] [3]

animalList = [“cat”, “dog”, “cow”, “big bird”]

[0] [1] [2] [3]

Access ‘reference’ to an item is via its index
Index always starts at [0] and are always integer – show them in brackets
In animalList: index [3] has two strings but is still one index